GIT, GESTION DE PROJET ET AGILITÉ · JOUR 5 SUR 5 · VENDREDI 30 OCTOBRE

Vendredi · TP : le dépôt de la promo

Aujourd'hui, toute la promo (Jonathan, Loubna, Richard et Rigobert) travaille dans **un seul dépôt GitHub**. Loubna est la cheffe de projet : elle crée le dépôt, invite tout le monde, tient le tableau et fusionne les pull requests. Les autres développent.

Le projet est volontairement petit : un script `promo.sh` qui présente chaque membre de l'équipe. Ce qui nous intéresse, c'est le chemin : un tableau Kanban pour se répartir le travail, une branche par personne, des commits, des PR, et des conflits. Beaucoup de conflits. Ils sont prévus, et vous allez les régler vous-mêmes.

Les règles du jeu

1. **Personne ne pousse sur `main`.** Tout passe par une pull request, que Loubna relit et fusionne.
2. **Chacun a sa branche**, à son prénom, en minuscules et sans accent : `jonathan`, `loubna`, `richard`, `rigobert`. Vous la gardez toute la journée.
3. **Avant chaque nouvelle tâche**, sur votre branche : `git pull origin main`. Vous récupérez ainsi le travail déjà fusionné des autres.
4. **Une tâche = une issue = une PR.** La description de la PR contient `Closes #numéro`, le numéro de l'issue.
5. **Le tableau dit la vérité.** Vous commencez une tâche : sa carte passe dans *In Progress*. La PR est fusionnée : elle passe dans *Done*.

Le déroulé

Heure	Étape	Qui
9 h 00	**0.** Préparer le dépôt, le tableau et les issues	Loubna. Les autres vérifient leur clé SSH et donnent leur pseudo GitHub à Loubna
9 h 45	**1.** Rejoindre le dépôt, créer sa branche	tout le monde
10 h 15	**2.** Ma fiche : un premier commit, sans conflit	tout le monde
11 h 15	**3.** Mon nom dans le README : le conflit garanti	tout le monde
14 h 00	**4.** Même ligne, deux versions : le conflit à deux	par binômes
15 h 30	**5.** Démo : `promo.sh` depuis `main`, le tableau, l'historique	Loubna, en partage d'écran
16 h 15	Bilan de la journée	tout le monde

Les horaires sont indicatifs. Si une étape prend plus de temps, on la termine avant de passer à la suivante.

Étape 0 · Loubna prépare le dépôt

Pendant ce temps, les autres tapent `ssh -T git@github.com` pour vérifier que leur clé SSH fonctionne (la réponse doit commencer par « Hi pseudo! »), puis envoient leur pseudo GitHub à Loubna.

1. Le squelette du projet. Sur sa machine, Loubna crée le dossier `promo-git` avec trois choses dedans : le `README.md`, le script `promo.sh`, et un dossier `membres/` qui contient un fichier vide `.gitkeep` (Git ne suit pas les dossiers vides).

README.md

```
# Promo Git · Prépavenir

Le dépôt de la promo pour le TP Git du vendredi 30 octobre 2026.
Pour présenter toute l'équipe : bash promo.sh

## L'équipe

- Loubna, cheffe de projet
```

promo.sh

```
#!/bin/bash
# Présente l'équipe : lance la fiche de chaque membre, par ordre alphabétique.

echo "=== Bienvenue ==="

for fiche in membres/*.sh; do
    [[ -f "$fiche" ]] || continue    # au début, le dossier est vide
    bash "$fiche"
    echo "---"
done

echo "Fin"
```

2. Le dépôt sur GitHub. Sur github.com : bouton **+** en haut à droite, **New repository**. Nom : `promo-git`, visibilité **Public**, et ne cochez rien d'autre (pas de README, il existe déjà sur votre machine). **Create repository**.

3. Le premier et dernier push sur `main`. C'est la seule exception à la règle 1 : il faut bien un premier commit sur `main` pour que les autres aient une base.

Loubna, sur sa machine

```
$ mkdir promo-git && cd promo-git
$ git init
Initialized empty Git repository in /home/robin/demo/promo-git/.git/
$ ls
README.md
membres
promo.sh
$ git add .
$ git commit -m "Crée le squelette du projet"
[main (root-commit) 88d44de] Crée le squelette du projet
 3 files changed, 20 insertions(+)
 create mode 100644 README.md
 create mode 100644 membres/.gitkeep
 create mode 100644 promo.sh
$ git remote add origin git@github.com:Loubna-Boudaoud/promo-git.git
$ git push -u origin main
To git@github.com:Loubna-Boudaoud/promo-git.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

4. Inviter l'équipe. Dans le dépôt : **Settings**, **Collaborators**, **Add people**, puis le pseudo de chaque personne. GitHub peut redemander le mot de passe de Loubna avant d'ouvrir cette page. Chaque invité reçoit un mail.

5. Le tableau. Dans le dépôt : onglet **Projects**, **New project**, modèle **Board**, nom `Promo Git`. Le tableau a trois colonnes : *Todo*, *In Progress*, *Done*.

6. Les issues. Une fois que les invitations sont acceptées (étape 1), et pas avant : GitHub ne propose d'assigner une issue qu'aux personnes qui ont rejoint le dépôt. Pour chaque membre de l'équipe, Loubna crée deux issues : onglet **Issues**, **New issue**, un titre, une description, puis dans la colonne de droite, **Assignees** (la personne) et **Projects** (`Promo Git`).

Titre de l'issue	Description	Assignée à
Fiche de Jonathan	Créer `membres/jonathan.sh`, qui affiche ton prénom et deux phrases sur la semaine.	Jonathan
Fiche de Loubna	Créer `membres/loubna.sh`, même chose.	Loubna
Fiche de Richard	Créer `membres/richard.sh`, même chose.	Richard
Fiche de Rigobert	Créer `membres/rigobert.sh`, même chose.	Rigobert
Jonathan dans le README	Ajouter `- Jonathan` à la fin de la liste « L'équipe » du README.	Jonathan
Richard dans le README	Ajouter `- Richard` à la fin de la liste « L'équipe » du README.	Richard
Rigobert dans le README	Ajouter `- Rigobert` à la fin de la liste « L'équipe » du README.	Rigobert

Loubna est déjà dans la liste du README : elle n'a pas d'issue à ce tour-là, ce qui lui laisse le temps de relire et fusionner. Les issues de l'étape 4 seront créées après la pause de midi. Si une carte arrive dans la colonne *No Status* du tableau, glissez-la dans *Todo*.



Étape 1 · Rejoindre le dépôt

1. **Acceptez l'invitation** : dans le mail reçu, ou directement à l'adresse `github.com/Loubna-Boudaoud/promo-git/invitations`. Sans cette étape, vous ne pourrez pas pousser.
2. **Clonez le dépôt** en SSH : sur la page du dépôt, bouton vert **Code**, onglet **SSH**, copiez l'adresse.
3. **Créez votre branche**, à votre prénom.

Richard, sur sa machine

```
$ git clone git@github.com:Loubna-Boudaoud/promo-git.git
Cloning into 'promo-git'...
done.
$ cd promo-git
$ git switch -c richard
Switched to a new branch 'richard'
$ git branch
  main
* richard
```

Loubna aussi crée sa branche, `loubna`, dans son dossier `promo-git`. À partir de maintenant, elle travaille comme les autres.

Étape 2 · Ma fiche

Ouvrez votre issue « Fiche de… » dans le tableau et glissez sa carte dans *In Progress*.

Créez votre fiche dans le dossier `membres/`, avec votre prénom en nom de fichier : `nano membres/richard.sh`. Le modèle :

membres/richard.sh

```
#!/bin/bash
# Fiche de Richard
echo "Richard"
echo "  Mon exercice préféré de la semaine bash : le pendu"
echo "  Ce que je retiens de Git : on ne pousse jamais sur main"
```

Testez avec `bash promo.sh`, puis commit et push :

Richard crée sa fiche

```
$ bash promo.sh
=== Bienvenue ===
Richard
  Mon exercice préféré de la semaine bash : le pendu
  Ce que je retiens de Git : on ne pousse jamais sur main
---
Fin
$ git add membres/richard.sh
$ git commit -m "Ajoute la fiche de Richard"
[richard a1b0b12] Ajoute la fiche de Richard
 1 file changed, 5 insertions(+)
 create mode 100644 membres/richard.sh
$ git push -u origin richard
To git@github.com:Loubna-Boudaoud/promo-git.git
 * [new branch]      richard -> richard
branch 'richard' set up to track 'origin/richard'.
```

Le `-u` ne sert qu'au premier push de la branche. Ensuite, un simple `git push` suffit.

Ouvrez la pull request. Sur GitHub, un bandeau jaune propose **Compare & pull request**. Vérifiez en haut : `base: main` ← `compare: richard`. Titre : « Ajoute la fiche de Richard ». Description : `Closes #3` (le numéro de votre issue, ici la troisième créée par Loubna). **Create pull request**.

Loubna relit et fusionne. Onglet **Files changed** pour lire le code, puis **Merge pull request**, **Confirm merge**. GitHub propose ensuite **Delete branch** : Loubna ne clique pas, chacun garde sa branche toute la journée. L'issue se ferme toute seule grâce au `Closes`, et sa carte passe dans *Done*.

Pourquoi aucun conflit ici ? Chacun a créé **son propre fichier**. Deux personnes qui ne touchent pas au même fichier ne peuvent pas se gêner.

Étape 3 · Mon nom dans le README

Cette fois, tout le monde modifie **le même fichier, au même endroit**. Le conflit est garanti, et c'est le but.

1. Carte « … dans le README » dans *In Progress*.
2. Sur votre branche : `git pull origin main`, pour récupérer les fiches des autres.
3. Ouvrez le README avec `nano README.md` et ajoutez `- votre prénom` **à la fin** de la liste « L'équipe ».

4. `git commit -am "Ajoute Richard à l'équipe"`, puis `git push`.
5. Ouvrez une **nouvelle** pull request. L'ancienne est fusionnée et fermée : GitHub reproposera **Compare & pull request** pour vos nouveaux commits.

La première PR que Loubna fusionne passe sans problème. Toutes les autres affichent alors : *This branch has conflicts that must be resolved*. Normal : votre ligne et celle de la personne fusionnée avant vous sont au même endroit, et Git ne sait pas laquelle mettre en premier.

Le conflit se règle sur votre machine, sur votre branche. Exemple avec Rigobert, qui arrive juste après Richard :

Rigobert récupère main… et le conflit

```
$ git pull origin main
From github.com:Loubna-Boudaoud/promo-git
 * branch            main       -> FETCH_HEAD
   0f378ca..3088468  main       -> origin/main
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
$ cat README.md
# Promo Git · Prépavenir

Le dépôt de la promo pour le TP Git du vendredi 30 octobre 2026.
Pour présenter toute l'équipe : bash promo.sh

## L'équipe

- Loubna, cheffe de projet
<<<<<<< HEAD
- Rigobert
=======
- Richard
>>>>>>> 30884680adfe0fd931a92bf23e2105cb4a9a2fa4
```

Entre `<<<<<<< HEAD` et `=======`, votre version. Entre `=======` et `>>>>>>>`, celle qui est déjà sur `main` (avec `git pull origin main`, Git l'étiquette par un identifiant de commit au lieu d'un nom de branche). Ici, on garde **les deux lignes** et on supprime les trois marqueurs.

> **Régler un conflit avec nano.** `nano README.md`, descendez avec les flèches jusqu'à une ligne de marqueur (`<<<<<<<`, `=======` ou `>>>>>>>`) et supprimez-la avec **Ctrl+K**. Recommencez pour les deux autres. Enregistrez avec **Ctrl+O** puis **Entrée**, quittez avec **Ctrl+X**.

Rigobert règle le conflit

```
$ tail -n 4 README.md

- Loubna, cheffe de projet
- Richard
- Rigobert
$ git add README.md
$ git commit --no-edit
[rigobert 51f0f67] Merge branch 'main' of github.com:Loubna-Boudaoud/promo-git into rigobert
$ git push
To git@github.com:Loubna-Boudaoud/promo-git.git
   a056e5f..51f0f67  rigobert -> rigobert
```

La PR se met à jour toute seule et le message de conflit disparaît. Loubna peut fusionner.

Si Loubna a fusionné une autre PR entre-temps, le conflit peut revenir. Même recette : `git pull origin main`, garder les lignes, `add`, `commit`, `push`. Le dernier fusionné le réglera peut-être deux fois : c'est lui qui aura le plus appris.

> **Avant chaque commit de résolution**, vérifiez qu'il ne reste aucun marqueur : `grep -rn "<<<<<<<" .` ne doit rien afficher. Et relancez `bash promo.sh` : un script qui contient encore des marqueurs ne marche plus.

Étape 4 · Même ligne, deux versions

À l'étape 3, chacun ajoutait une ligne : il suffisait de garder les deux. Ici, deux personnes modifient **la même ligne**, et le script ne doit en garder qu'une. Il faudra se parler.

Après la pause, Loubna crée deux issues et assigne chacune à **deux personnes** :

Titre de l'issue	La ligne à modifier dans `promo.sh`	Binôme
Message d'accueil	`echo "=== Bienvenue ==="`	Richard et Rigobert
Message de fin	`echo "Fin"`	Jonathan et Loubna

Richard est à distance : son binôme et lui se parlent en visio ou par message, c'est comme ça que ça se passe dans beaucoup d'équipes.

1. Chacun de son côté, **sans en parler à son binôme** : `git pull origin main`, modifiez la ligne à votre goût avec `nano promo.sh`, testez avec `bash promo.sh`, commit, push, pull request.
2. Loubna fusionne la première PR qui arrive. Celle du binôme passe en conflit.
3. La personne en conflit fait `git pull origin main`, puis va voir son binôme. Ensemble, vous écrivez **une seule ligne** qui mélange vos deux idées. Pas deux lignes l'une sous l'autre : le script ne doit afficher qu'un message.

Exemple : Richard a remplacé l'accueil par `=== Bienvenue dans la promo ===`, Rigobert par `=== Salut l'équipe ===`. La PR de Richard est fusionnée en premier.

Rigobert, en conflit avec Richard sur la même ligne

```
$ git pull origin main
From github.com:Loubna-Boudaoud/promo-git
 * branch            main       -> FETCH_HEAD
   7170611..a2b87a3  main       -> origin/main
Auto-merging promo.sh
CONFLICT (content): Merge conflict in promo.sh
Automatic merge failed; fix conflicts and then commit the result.
$ head -n 9 promo.sh
#!/bin/bash
# Présente l'équipe : lance la fiche de chaque membre, par ordre alphabétique.

<<<<<<< HEAD
echo "=== Salut l'équipe ==="
=======
echo "=== Bienvenue dans la promo ==="
>>>>>>> a2b87a3a419e8857ebf6c04b33be4dc5ab4e81bb
```

Après discussion, Rigobert remplace les cinq lignes du conflit par une seule : `echo "=== Salut l'équipe, bienvenue dans la promo ==="`.

Rigobert règle le conflit

```
$ bash promo.sh
=== Salut l'équipe, bienvenue dans la promo ===
Loubna
  Mon exercice préféré de la semaine bash : le palindrome
  Ce que je retiens de Git : un conflit, ça se règle en se parlant
---
Richard
  Mon exercice préféré de la semaine bash : le pendu
  Ce que je retiens de Git : on ne pousse jamais sur main
---
Rigobert
  Mon exercice préféré de la semaine bash : la moyenne des notes
  Ce que je retiens de Git : git status, tout le temps
---
Fin
$ git add promo.sh
$ git commit --no-edit
[rigobert 1ab4453] Merge branch 'main' of github.com:Loubna-Boudaoud/promo-git into rigobert
$ git push
To git@github.com:Loubna-Boudaoud/promo-git.git
   2b302b8..1ab4453  rigobert -> rigobert
```

Étape 5 · La démo

Tout le monde revient sur `main` et récupère la version finale :

Loubna, en partage d'écran

```
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git pull
From github.com:Loubna-Boudaoud/promo-git
   88d44de..a3edacc  main       -> origin/main
 * [new branch]      richard    -> origin/richard
 * [new branch]      rigobert   -> origin/rigobert
Updating 88d44de..a3edacc
Fast-forward
 README.md           | 2 ++
 membres/loubna.sh   | 5 +++++
 membres/richard.sh  | 5 +++++
 membres/rigobert.sh | 5 +++++
 promo.sh            | 2 +-
 5 files changed, 18 insertions(+), 1 deletion(-)
 create mode 100644 membres/loubna.sh
 create mode 100644 membres/richard.sh
 create mode 100644 membres/rigobert.sh
$ bash promo.sh
=== Salut l'équipe, bienvenue dans la promo ===
Loubna
  Mon exercice préféré de la semaine bash : le palindrome
  Ce que je retiens de Git : un conflit, ça se règle en se parlant
---
Richard
  Mon exercice préféré de la semaine bash : le pendu
  Ce que je retiens de Git : on ne pousse jamais sur main
---
Rigobert
  Mon exercice préféré de la semaine bash : la moyenne des notes
  Ce que je retiens de Git : git status, tout le temps
---
Fin
```

Remarquez le message de `git switch main` : *up to date with 'origin/main'*. Git compare avec ce qu'il sait de GitHub depuis le dernier échange, pas avec GitHub en direct. D'où le `git pull` juste après, qui récupère d'un coup tout le travail de la journée.

Puis Loubna montre :

- **le tableau** : toutes les cartes dans *Done* ;
- **l'historique** : `git log --oneline --graph`, ou sur GitHub, onglet **Insights**, **Network**. Chaque bosse du graphe, c'est une branche qui revient dans `main` ;
- **qui a fait quoi** : `git shortlog -sn HEAD`, le nombre de commits par personne.

Pour finir en beauté, Loubna pose une étiquette de version sur `main` : `git tag -a v1.0 -m "Version de fin de TP"`, puis `git push origin v1.0`. Sur GitHub, elle apparaît dans **Tags**.

Le bilan

Chacun répond en une phrase, à l'oral ou dans une issue « Bilan » que Loubna ouvre pour l'occasion :

- Qu'est-ce qui a bien marché ?
- Qu'est-ce qui vous a bloqué, et comment vous en êtes sortis ?
- Que feriez-vous autrement dans un vrai projet d'équipe ?

Vous avez fini en avance ?

- **Relisez une PR ouverte** de quelqu'un d'autre : onglet **Files changed**, cliquez sur le **+** à côté d'une ligne pour laisser un commentaire.
- **Créez vous-même une issue** « Améliorer ma fiche », assignez-la-vous, ajoutez une ligne à votre fiche et faites tout le circuit : carte, branche, commit, PR.
- **Retrouvez dans l'historique** le commit qui a ajouté votre fiche : `git log --oneline -- membres/votre-prenom.sh`.
- Pas encore joué ? *L'enquête* et *Le festival* vous attendent.

Les commandes de la journée

Commande	À quoi elle sert aujourd'hui
`git clone <adresse>`	récupérer le dépôt de Loubna
`git switch -c prenom`	créer sa branche, une seule fois le matin
`git branch`	vérifier qu'on est bien sur sa branche (l'étoile)
`git pull origin main`	récupérer le travail fusionné, avant chaque tâche et pour régler un conflit
`git add` puis `git commit -m "…"`	enregistrer son travail
`git push -u origin prenom`	le premier envoi de sa branche
`git push`	les envois suivants
`git commit --no-edit`	terminer une résolution de conflit avec le message proposé
`grep -rn "<<<<<<<" .`	vérifier qu'il ne reste aucun marqueur de conflit
`git log --oneline --graph`	voir les branches revenir dans `main`